

Intel 8088 Bus-Compatible Memory & I/O

SystemVerilog modeling of an Intel 8088 microprocessor bus interface

Sasha Katne

PSU ECE — Introduction to SystemVerilog

github.com/sashakatne/8088BusCompatibleModules

The Problem

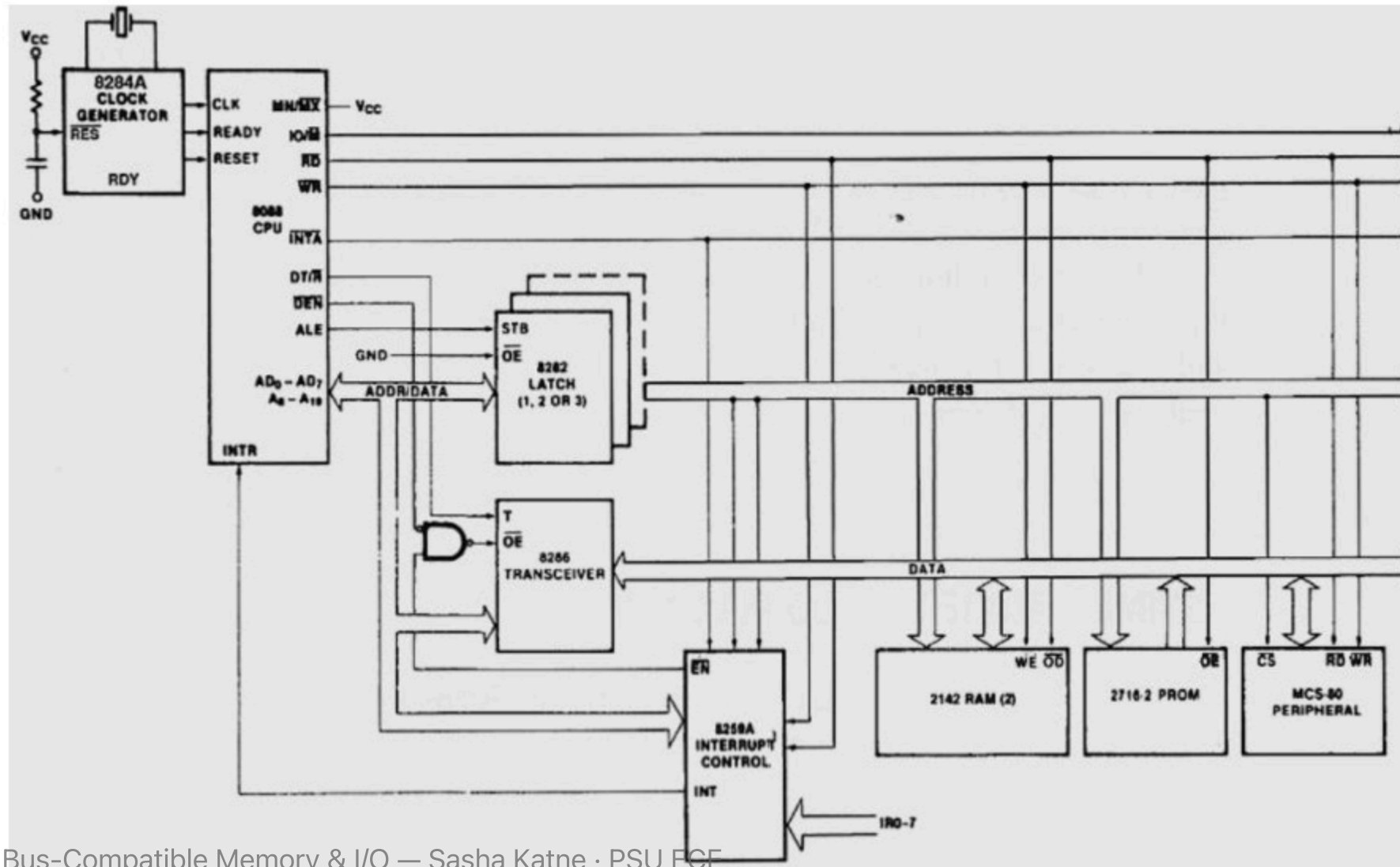
The Intel 8088 has a **multiplexed address/data bus**: a 20-bit address and an 8-bit data byte share the same physical pins.

A compatible peripheral must:

- **Latch** the lower address byte from `AD[7:0]` on the **ALE** strobe (the upper bits `A[19:8]` arrive separately)
- Distinguish **memory vs. I/O** access via the `IOM` pin
- Respond to active-low `RD` / `WR` strobes at the right phase
- **Tristate** the data bus when not driving it

This project models the **peripheral side** — two 512 KB memory banks and two I/O port ranges — as parameterized SystemVerilog modules that interoperate with an encrypted Intel 8088 reference IP.

System Architecture



MemoryOrIOModule — One Module, Four Roles

```
module MemoryOrIOModule (Intel8088Pins bus, input wire CS);  
    parameter ADDR_WIDTH = 20;  
    parameter DATA_WIDTH = 8;  
    parameter INIT_FILE = "memory_init.mem";  
  
    Datapath          datapath (...); // memory array + tristate  
    ControlSequencer  ctrl      (...); // FSM  
endmodule
```

Instance	Width	Address range	Init file
M0	19 bits	0x00000 – 0x7FFFF	memory0_init.mem
M1	19 bits	0x80000 – 0xFFFFF	memory1_init.mem
I00	16 bits	0xFF00 – 0xFF0F	io_device0_init.mem
I01	16 bits	0x1C00 – 0x1DFF	io_device1_init.mem

DRY win: same RTL covers two address-space sizes and two functional roles.

Inside the Module

Datapath — memory array + tristate driver

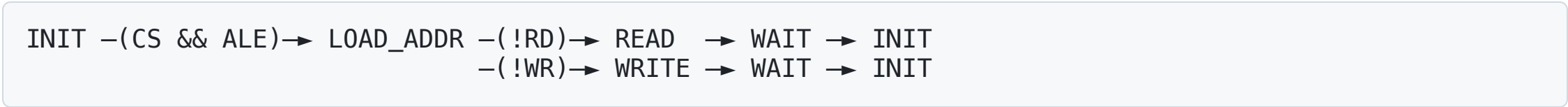
```
reg [DATA_WIDTH-1:0] MEM[NUM_UNITS-1:0];
assign DATA = OE ? MEM[ADDR_REG] : 'z;

always_ff @(posedge CLK) begin
    if (LA) ADDR_REG <= ADDRESS;           // capture address
    if (WE) MEM[ADDR_REG] <= DATA;        // capture write data
end
```

ControlSequencer — clocked FSM

- Inputs: ALE, RD, WR, CS (from the 8088 / top-level decode)
- Outputs: LA, OE, WE (back to Datapath)

Strict separation of concerns — Datapath is dumb storage; the FSM is the only stateful logic.



Address Decode + Bus Wrappers

Chip-select decode — `tb/top_interface.sv`

```
M0_CS = ~IOM & ~Address[19];           // low 512 KB
M1_CS = ~IOM & Address[19];             // high 512 KB
I00_CS = IOM & ((Address[15:0] & 16'hFFF0) == 16'hFF00); // 16 ports
I01_CS = IOM & ((Address[15:0] & 16'hFE00) == 16'h1C00); // 512 ports
```

8282 transparent address latch

```
always_latch begin
    if (bus.ALE) bus.Address <= {bus.A, bus.AD};
end
```

8286 bidirectional data transceiver

```
assign bus.Data = ( bus.DTR & ~bus.DEN) ? bus.AD   : 'z;
assign bus.AD   = (~bus.DTR & ~bus.DEN) ? bus.Data : 'z;
```

External glue modeled at top level — no extra modules.

Verification — Two Independent Paths

Path 1: Encrypted Intel 8088 IP (`ip/8088if.svp`)

- QuestaSim-encrypted reference 8088 model
- Driven by a scripted trace `stimulus/busops.txt`:

```
12 M W 0x33333      # at sim-time 12, memory write to 0x33333
```

- **Proof:** reaches `$finish` after 300 clocks; waveforms align with the 8088 timing diagram in `docs/images/wr_rd_timing.jpg`

Path 2: Self-checking testbench (`tb/memorio_tb.sv`)

- Authored directed stimulus + golden expected values
- Sweeps **1024+1024 memory ops** and **16+512 I/O ops** in each direction
- **Proof:** transcript ends in `*** PASSED ***` with zero `Error: Read data ... does not match ...` lines

Self-Checking Testbench

```
task automatic WriteOperation(input [19:0] addr, input [7:0] data, input iom);
    bus.IOM = iom;
    bus.A   = addr[19:8];
    bus.AD  = addr[7:0];
    bus.ALE = '1; #50; bus.ALE = '0;          // pulse ALE, latch address
    bus.AD  = data;
    bus.WR  = '0; #100; bus.WR = '1;          // pulse write strobe
endtask
```

ReadOperation mirrors it with RD and compares the captured data against the golden array.

```
for (int a = 0; a < 1024; a++) WriteOperation(a, golden[a], MEM_ACCESS);
for (int a = 0; a < 1024; a++) ReadOperation (a, golden[a], MEM_ACCESS);
// ... I/O ranges ...
$display(Error ? "*** FAILED ***" : "*** PASSED ***");
```

Task-based bus-functional model — clean, reusable, fully automated verdict.

Results

Functional verdict (all three configurations)

- Moore + IP path: clean `$finish`, all bus ops in trace consumed
- Moore + self-checking: `*** PASSED ***` (~3000 directed ops)
- Mealy + self-checking: `*** PASSED ***` (~3000 directed ops)

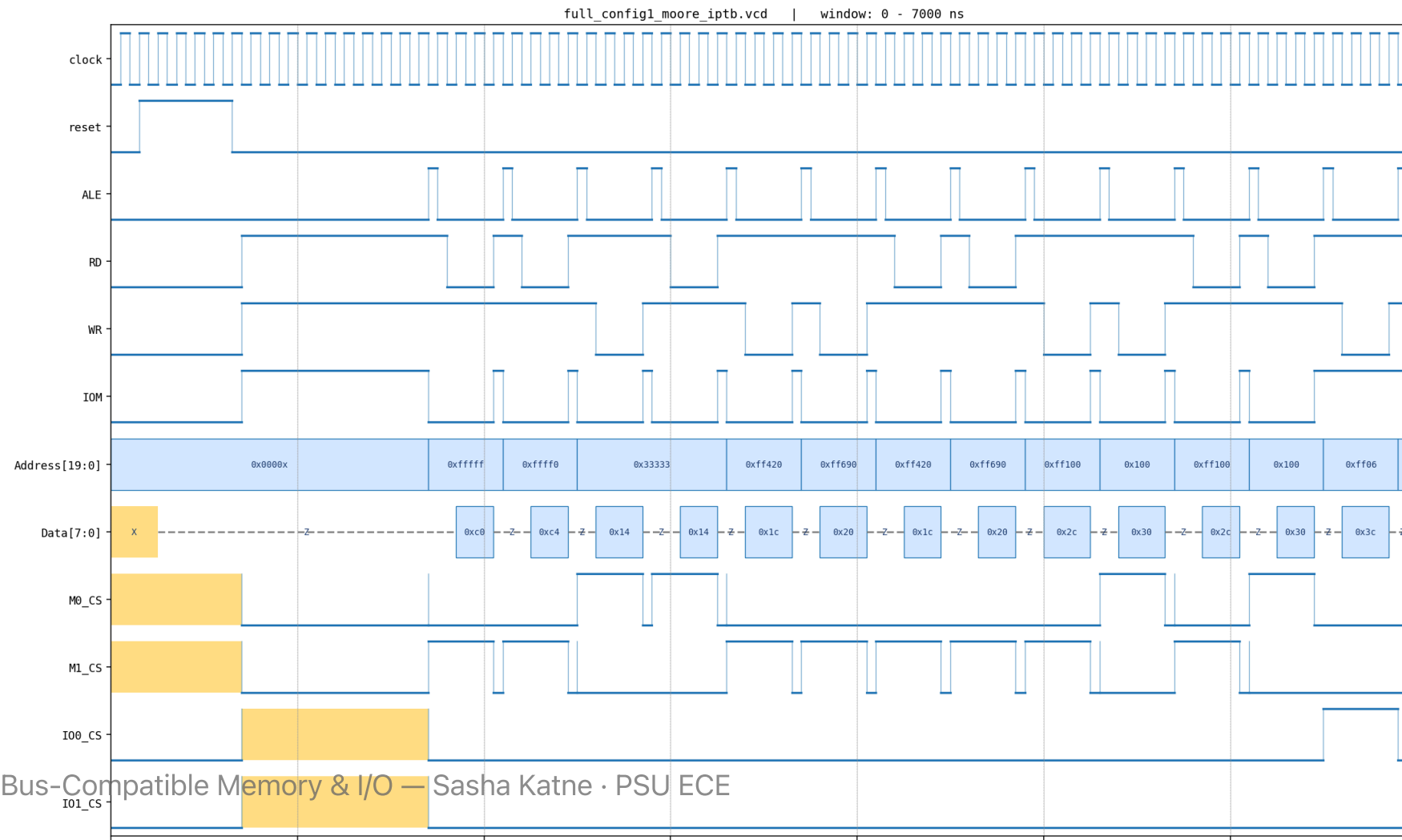
Coverage (Moore + IP, instance-scoped to `MemoryOrIOModule(rtl)`)

Metric	Result
FSM states	5 / 5 (100%)
FSM transitions	6 / 9 (67%)
Statements	15 / 15 (100%)
Aggregate (by instance)	93.33%

Tracked evidence

Engineering Insight #1 — Combinational Decode Hazard

The matplotlib waveform shows narrow glitch pulses on `M0_CS` / `M1_CS` between bus operations.



Engineering Insight #2 — Remote Verification Caught a Real Bug

After repo reorganization, I ran the full suite on the PSU ECE farm to verify nothing broke. The first remote run produced a `vlib-34 Library already exists` warning.

Diagnosis: `scripts/run.do` had `catch {vdel -all}` **before** `cd sim`, so `vdel` was trying to delete a non-existent `work/` in the parent directory while `sim/work/` from the previous run survived.

```
- catch {vdel -all}
- cd sim
- vlib work
+ cd sim                # change CWD first
+ catch {vdel -all}      # so vdel hits sim/work/
+ vlib work
```

Lesson: local "it ran clean" masked a real defect because my local repo was a fresh checkout with no stale library. The project's verification gate — "**Errors: 0, Warnings: 0**" — plus remote-farm rerun caught what casual single-host testing missed.

Tooling & Workflow

Simulation

- QuestaSim 2021.3_1 — `vlog / vopt / vsim / vcover` toolchain
- Coverage scoped to `+cover=sbfec+Memory0rIOModule(rtl).` — DUT only, not the testbench harness

Remote verification pipeline (PSU ECE farm)

- SSH alias `katnemo` → `mo.ece.pdx.edu`
- Per-run isolation: `~/claude-runs/<UTC-stamp>_<slug>/`
- Every run logs `MANIFEST.txt` with command, start/end UTC, pass verdict, caveats

Evidence-as-code

- `docs/sim_evidence/` is **tracked** in git: transcripts, UCDBs, PNG waveforms
- Source `.wlf` / `.vcd` files intentionally **not** committed (large, regenerable)
- A reviewer can read a known-good baseline without re-running the sim

Summary

What this project demonstrates

- **SystemVerilog idioms:** `interface` + `modport`, `always_ff`/`always_comb`/`always_latch`, parameterized modules, typed enums for FSM state, tasks for BFM stimulus
- **Bus-protocol modeling:** 8088 multiplexed AD bus, 8282 address latch, 8286 data transceiver, `IOM` / `RD` / `WR` sequencing
- **FSM design:** Moore (5 states) **and** Mealy (3 states) variants of the same controller; both verified
- **Verification rigor:** dual-path verification (encrypted IP + self-checking BFM), coverage scoping, tracked sim evidence
- **Engineering discipline:** caught a real `run.do` defect via remote re-verification; documented a non-obvious combinational hazard rather than hiding it

Key files

- `rtl/memorio.sv` · `rtl/memorio_mealy.sv` · `rtl/interface.sv`

- `tb/top_interface.sv` · `tb/memorio_tb.sv`

- `scripts/run.do` · `scripts/memfilegenerator.py` · `scripts/ved_to_png.py`

Questions?

Happy to go deep on:

- FSM design tradeoffs (Moore vs. Mealy here)
- Verification strategy and coverage scoping
- The chip-select glitch hazard
- The remote-farm verification pipeline
- Any specific signal or transition in the waveforms

github.com/sashakatne/8088BusCompatibleModules